Challenge 2:

Karim Negm - 11063520

Challenge Question 1:

How many different Confirmable PUT requests obtained an unsuccessful response from the local CoAP server?

ANSWER: 22

Answer motivation:

As there were too many packets to analyze manually, a python script was used the following logic:

- **Pass 1 - Identify Requests:**
  - Scans the challenge2.pcapng file for all CoAP packets that are:
    - CON type (needs acknowledgment).
    - PUT method.
    - Sent *to* the local_server_ip.

```
display_filter = 'coap && coap.code == 3 && coap.type == 0 && ip.dst == 127.0.0.1 && !_ws.malformed'
```

  - Stores a mapping of each request's Message ID (MID) to its Token.
- **Pass 2 - Identify Failures:**
  - Scans the pcapng file again for CoAP packets that are:
    - Sent *from* the local_server_ip.
    - Indicate failure: either a Reset (RST) type message OR any message with an error code (>= 128, like 4.xx or 5.xx).

```
display_filter='coap && ip.src == 127.0.0.1 && (coap.code >= 128 || coap.type == 3) && !_ws.malformed '
```

- **Correlation & Counting:**
  - For each failure response found, it tries to match it back to an original request using either the MID or the Token.
  - If a match is found, the Token of the *original failed request* is added to a set. Using a set automatically handles uniqueness – duplicate tokens (from retransmissions or multiple error packets for one request) are only stored once.
- **Result:** The final count is the total number of unique Tokens collected in the set, representing the distinct number of CON PUT requests that failed.

```python
import pyshark

pcap_file = 'challenge2.pcapng'
local_server_ip = '127.0.0.1'

request_info = {} # Stores {MID: Token} of CON PUT requests
failed_request_tokens = set() # Stores unique tokens of requests that failed

# Pass 1: Find all CON PUT requests, map MID -> Token
cap1 = pyshark.FileCapture(pcap_file, display_filter=f'coap && coap.code == 3
&& coap.type == 0 && ip.dst == {local_server_ip} && !_ws.malformed')
for req_packet in cap1:
    request_info[req_packet.coap.mid] = req_packet.coap.token
cap1.close()

original_request_tokens = set(request_info.values()) # Helper set for token
lookup

# Pass 2: Find unsuccessful responses (Error code >= 128 OR RST type 3) FROM
server
cap2 = pyshark.FileCapture(pcap_file, display_filter=f'coap && !_ws.malformed
&& ip.src == {local_server_ip} && (coap.code >= 128 || coap.type == 3)')
for rsp_packet in cap2:
    response_mid = rsp_packet.coap.mid
    response_token = rsp_packet.coap.token
    response_type = int(rsp_packet.coap.type)
    response_code = int(rsp_packet.coap.code)

    if response_mid in request_info: # Check 1: MID match (RST or direct ACK
error)
        failed_request_tokens.add(request_info[response_mid]) # Add original
request's token
    elif response_type != 3 and response_code >= 128 and response_token in
original_request_tokens: # Check 2: Token match for separate errors
         failed_request_tokens.add(response_token) # Add this response's
token (which matches an original request)
cap2.close()

# Final Count is the number of unique tokens associated with failed requests
final_count = len(failed_request_tokens)
# Final output
print(f"\n--- RESULT ---")
print(f"CQ1: Found {final_count} different tokens of CON PUT requests that
failed (either by error code or RST).")
```

Challenge Question 2:

How many CoAP resources in the coap.me public server received the same number of unique Confirmable and Non Confirmable GET requests?

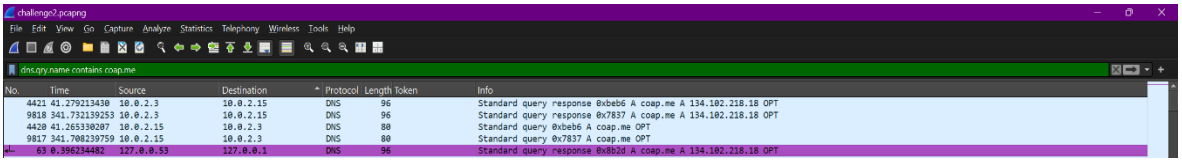
ANSWER: 3

- Path: large, CON: 14, NON: 14
- Path: validate, CON: 1, NON: 1
- Path: secret, CON: 1, NON: 1

Answer Motivation:

```
coap && ip.dst == 134.102.218.18 && coap.code == 1 && (coap.type == 0 || coap.type == 1) && !_ws.malformed'
```

- **Filtering**: Uses pyshark to read challenge2.pcapng, specifically selecting packets that are:
  - CoAP protocol.
  - Are GET requests (coap.code == 1).
  - Are either CON (coap.type == 0) or NON (coap.type == 1).
  - ip.dst == 134.102.218.18: Destination IP must be the coap.me server.
    - Found using dns.qry.name contains coap.me



- **Counting per Resource:**
  - Uses a defaultdict to store counts. The keys are the CoAP resource paths extracted from the Uri-Path option in the requests.
  - For each matching packet, it increments either a 'CON' counter or a 'NON' counter associated with that specific resource path.
- **Comparison:**
  - After processing all filtered packets, the script iterates through each resource path recorded in the counts.
  - It checks if the total count for 'CON' requests is equal to the total count for 'NON' requests for that path.
  - It also ensures this equal count is greater than zero (to exclude paths that received zero requests of both types).
  - Result: The final output is the total number of resource paths that satisfied the condition of having equal, non-zero counts of CON and NON GET requests.

```python
import pyshark
from collections import defaultdict

# --- Config ---
pcap_file = 'challenge2.pcapng'
coap_me_ip = "134.102.218.18"
coap_filter = f'coap && ip.dst == {coap_me_ip} && coap.code == 1 && (coap.type ==
0 || coap.type == 1) && !_ws.malformed'

# defaultdict to store counts of CON and NON requests for each path
# The default value is a dictionary with keys 'CON' and 'NON' initialized to 0
# This allows us to increment the counts easily without checking for existence
first
counts = defaultdict(lambda: defaultdict(int))
cap = pyshark.FileCapture(pcap_file, display_filter=coap_filter,
keep_packets=False)
for packet in cap:
    # Path extraction
    path = packet.coap.opt_uri_path
    # Increment CON (0) or NON (1) count
    counts[path]['CON' if int(packet.coap.type) == 0 else 'NON'] += 1

# --- Output ---
final_match_count = 0
# Find and print paths with equal CON and NON counts
for path, c in counts.items():
    if c.get('CON', 0) == c.get('NON', 0) and c.get('CON', 0) > 0:
        print(f"Path: {path}, CON: {c['CON']}, NON: {c['NON']}")
        final_match_count += 1
# --- Output ---
print(f"\n--- RESULT ---")
print(f"CQ2: Found {final_match_count} different resources that received the same
number of CON and NON requests.")
```

Challenge Question 3:

How many different MQTT clients subscribe to the public broker HiveMQ using multi-level wildcards?

Answer: 4

Answer Motivation:

- **Configuration:**
  - Specifies the input challenge2.pcapng file.
  - Lists the known IP addresses
    for broker.hivemq.com: 35.158.43.69, 35.158.34.213, 18.192.151.104.
    - From dns.qry.name contains hivemq
  - Initializes an empty set to store unique client identifiers (source IP and port).
- **Filtering:** ‘mqtt && (ip.dst == IP1 || ip.dst == IP2 || ...) && mqtt.msgtype == 8’
  - mqtt: Must be MQTT protocol.
  - (ip.dst == IP1 || ip.dst == IP2 || ...): Destination IP must match one of the predefined HiveMQ broker IPs.
  - mqtt.msgtype == 8: Only MQTT SUBSCRIBE messages.
- **Processing Logic:**
  - For each captured SUBSCRIBE packet:
    - Extracts the subscription topic.
    - Checks if the topic string ends with the multi-level wildcard character (#).
    - If it ends with #, the client's unique identifier (a tuple of source IP and source TCP port) is added to the set. The set automatically handles duplicates, ensuring each unique client is counted only once.
- **Result:** The final count is the total number of unique client identifiers (IP, Port pairs) collected in the set, representing the distinct clients using multi-level wildcard subscriptions to the specified HiveMQ IPs.

```python
import pyshark

# --- Configuration ---
pcap_file = 'challenge2.pcapng'

# IPs identified from DNS responses for broker.hivemq.com in the pcap
hivemq_ips = ["35.158.43.69", "35.158.34.213", "18.192.151.104"]

clients_using_multilevel = set() # Set to store unique clients using multi-level
wildcards

# --- Filtering and Processing ---
# Construct the IP filter part dynamically
ip_filter_part = " || ".join([f"ip.dst == {ip}" for ip in hivemq_ips])

# Filter for MQTT SUBSCRIBE (8) packets TO any of the HiveMQ broker IPs/Port
mqtt_filter = f'mqtt && ({ip_filter_part}) && mqtt.msgtype == 8'

print(f"Analyzing {pcap_file} for MQTT SUBSCRIBE to HiveMQ IPs ({',
'.join(hivemq_ips)})...")
print(f"Using filter: {mqtt_filter}")

processed_packets = 0
cap = pyshark.FileCapture(pcap_file, display_filter=mqtt_filter,
keep_packets=False)

for packet in cap:
    processed_packets += 1
    topic = packet.mqtt.topic
    if topic.endswith('#'):
        print(f"Packet {processed_packets}: Found SUBSCRIBE with multi-level
wildcard '#' in topic '{topic}' from IP {packet.ip.src} with
{packet.tcp.srcport}")
        clients_using_multilevel.add((packet.ip.src, packet.tcp.srcport))
print(f"\nProcessing complete. Analyzed {processed_packets} relevant SUBSCRIBE
packets.")

# --- Final Count ---
final_count = len(clients_using_multilevel)

# --- Output ---
print(f"\n--- RESULT ---")
print(f"CQ3: Found {final_count} different MQTT clients (unique source IP + Port)
subscribing to HiveMQ (IPs: {', '.join(hivemq_ips)}) using multi-level wildcards
('#').")
```

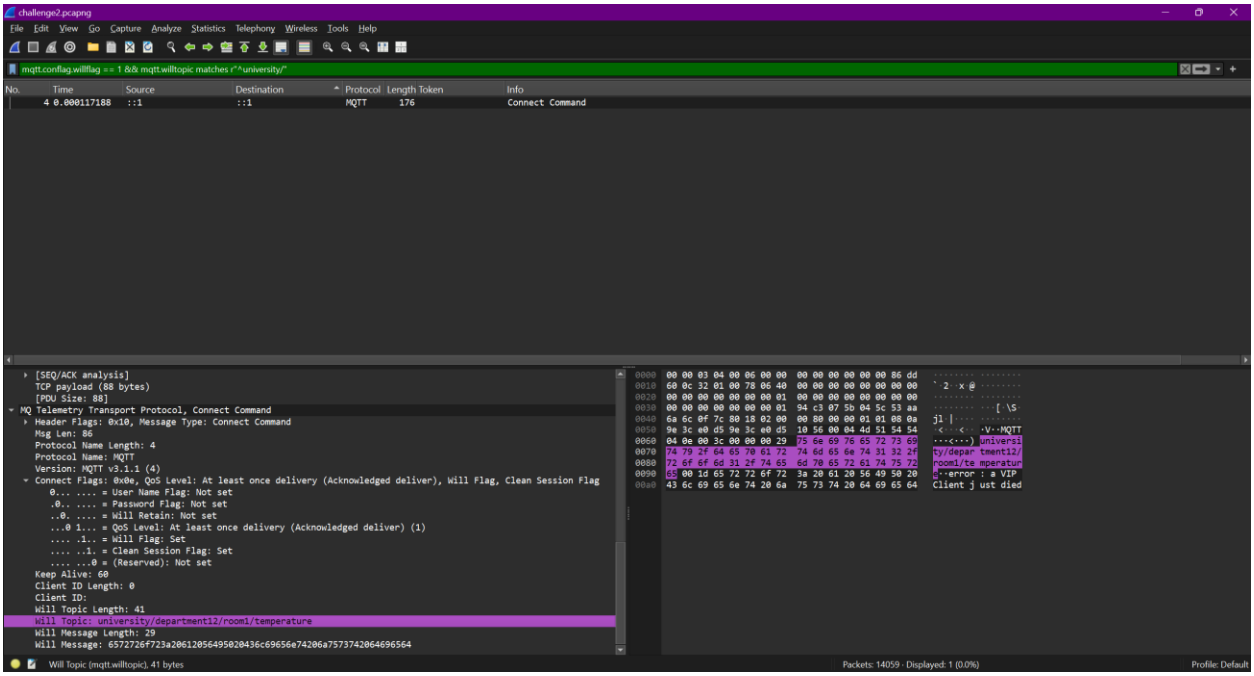
Challenge Question 4:

How many different MQTT clients specify a last Will Message to be directed to a topic having as first level "university"?

Answer: 1

Answer Motivation:

- using filter: mqtt.conflag.willflag == 1 && mqtt.willtopic matches r"^university/"
    - r-strings allow us to use regex to match for a string beginning with ‘university’
- We find only 1 client whose LWT is “university/department12/room1/temperature”



Challenge Question 5:

How many MQTT subscribers receive a last will message derived from a subscription without a wildcard?

Answer: 3

- Client ('::1', '39551') received message on last will topic:'university/department12/room1/temperature'.
- Client ('::1', '41789') received message on last will topic:'university/department12/room1/temperature'.
- Client ('::1', '53557') received message on last will topic:'university/department12/room1/temperature'.

Answer Motivation:

- **Goal:** Count unique MQTT clients (identified by IP + Port) that *received* a published message whose topic was designated as a Last Will Topic (LWT), specifically where the client's *original subscription* matching that topic *did not* contain any wildcards (# or +).
- **Pass 1: Identify Last Will Topics:**
  - **Filter:** 'mqtt.conflag.willflag == 1'
  - Filters for MQTT packets where the "Will Flag" is set (mqtt.conflag.willflag == 1).
  - Extracts the mqtt.willtopic from these packets.
  - Stores all unique LWTs found in a set (lwt_topics).
- **Pass 2: Identify Non-Wildcard Subscriptions:**
  - **Filter:** 'mqtt.msgtype == 8 && !_ws.malformed'
  - Filters for MQTT SUBSCRIBE messages (mqtt.msgtype == 8).
  - Extracts the subscriber's source IP, source port, and the mqtt.topic they are subscribing to.
  - Checks if the subscription topic contains *neither* # nor +.
  - Stores unique tuples of (subscriber_ip, subscriber_port, non_wildcard_topic) in a set (non_wildcard_subscription_tuples).
- **Pass 3: Correlate Published LWTs with Subscribers:**
  - **Filter:** 'mqtt.msgtype == 3 && !_ws.malformed'
  - Filters for MQTT PUBLISH messages (mqtt.msgtype == 3).

- For each PUBLISH message, checks two conditions:
    1. Is the mqtt.topic of the published message present in the lwt_topics set found in Pass 1? (i.e., is this message actually an LWT being published?)
    2. If it *is* an LWT, does any tuple in non_wildcard_subscription_tuples (from Pass 2) have a subscribed_topic that exactly matches this published topic?
  - If both conditions are met, it means a client with a non-wildcard subscription received a relevant LWT message. The unique identifier (subscriber_ip, subscriber_port) of that client is added to the final set (lwt_receivers_ip_port).

- **Result:** The final count is the number of unique client identifiers collected in the lwt_receivers_ip_port set, representing distinct clients that met all the specified conditions.

```
import pyshark

pcap_file = 'challenge2.pcapng'

# --- Pass 1: Identify all Last Will Topics ---
lwt_topics = set()
print("--- Pass 1: Identifying Last Will Topics ---")
lwt_filter = 'mqtt.conflag.willflag == 1'
lwt_cap = pyshark.FileCapture(pcap_file, display_filter=lwt_filter,
keep_packets=False)
for packet in lwt_cap:
    lwt_topics.add(str(packet.mqtt.willtopic))
    print(f"Found LWT topic: {packet.mqtt.willtopic}")
lwt_cap.close()

# --- Pass 2: Identify non-wildcard subscriptions (Client IP+Port, Topic) ---
# Store tuples: (subscriber_ip, subscriber_port, subscribed_topic)
non_wildcard_subscription_tuples = set()
print("\n--- Pass 2: Identifying Non-Wildcard Subscriptions by Client (IP+Port) -
--")
# Filter for SUBSCRIBE messages only
sub_filter = 'mqtt.msgtype == 8 && !_ws.malformed'
sub_cap = pyshark.FileCapture(pcap_file, display_filter=sub_filter,
keep_packets=False)
for packet in sub_cap:
    subscriber_ip = packet.ip.src if 'IP' in packet else packet.ipv6.src
    subscriber_port = packet.tcp.srcport
    topic = str(packet.mqtt.topic)
    # Check if the *subscription* is non-wildcard
    if '#' not in topic and '+' not in topic:
        subscription_tuple = (subscriber_ip, subscriber_port, topic)
```

```
        non_wildcard_subscription_tuples.add(subscription_tuple)
sub_cap.close()

# --- Pass 3: Find PUBLISH messages matching LWTs and correlate with subscriber
(IP+Port) ---
# Stores unique client identifiers (ip, port) that received a relevant LWT
lwt_receivers_ip_port = set()
print("\n--- Pass 3: Correlating PUBLISH Messages with LWTs and Subscriptions ---
")
# Filter for PUBLISH messages only
pub_filter = 'mqtt.msgtype == 3 && !_ws.malformed'
pub_cap = pyshark.FileCapture(pcap_file, display_filter=pub_filter,
keep_packets=False)
for packet in pub_cap:
    publish_topic = str(packet.mqtt.topic)
    # Condition 1: Is the published topic a Last Will Topic?
    if publish_topic in lwt_topics:
        # Condition 2: Find matching non-wildcard subscription(s) for this topic
        for sub_ip, sub_port, sub_topic in non_wildcard_subscription_tuples:
            if sub_topic == publish_topic:
                client_id = (sub_ip, sub_port)
                if client_id not in lwt_receivers_ip_port: # Avoid duplicate
prints for same client
                    print(f"MATCH: Client {client_id} received message on last
will topic:'{publish_topic}'.")
                lwt_receivers_ip_port.add(client_id)
pub_cap.close()

# --- Final Count ---
final_count = len(lwt_receivers_ip_port)
# --- Output ---
print(f"\n--- RESULT ---")
print(f"Found {final_count} unique MQTT client connections (unique source IP +
source Port) that RECEIVED a Last Will message derived from a subscription
WITHOUT a wildcard.")
```

Challenge Question 6:

How many MQTT publish messages directed to the public broker mosquitto are sent with the retain option and use QoS “At most once”?

Answer: 208

Answer Motivation:

Using filter:

- mqtt.msgtype == 3 &&(ip.dst == 5.196.78.28 || ipv6.dst == 2001:41d0:a:6f1c::1) && mqtt.retain == 1 && mqtt.qos == 0 && !_ws.malformed

Where:

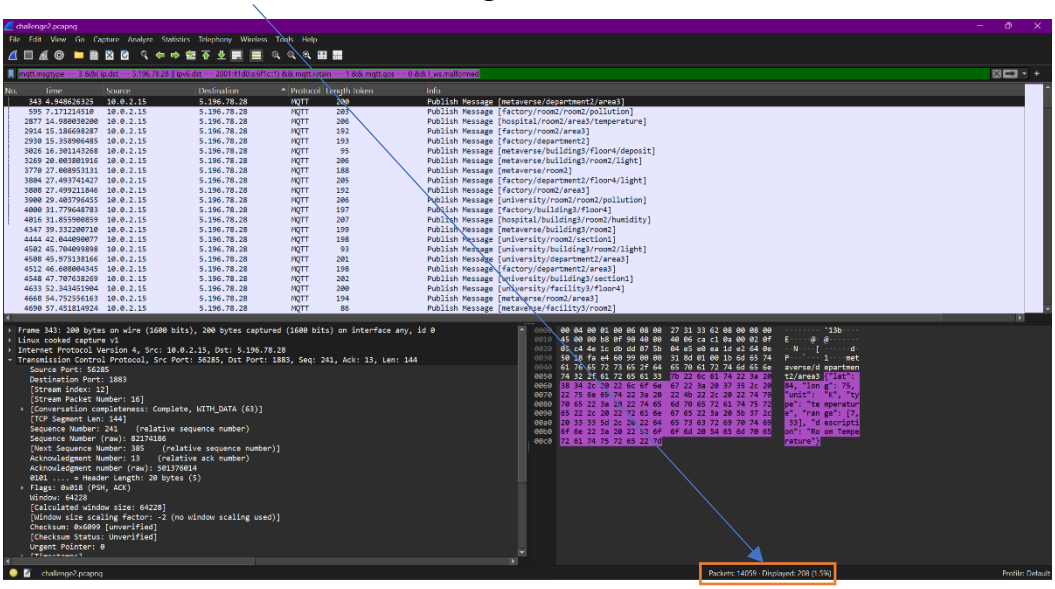
  - mqtt.msgtype == 3: Selects only MQTT PUBLISH messages.
  - (ip.dst == 5.196.78.28 || ipv6.dst == 2001:41d0:a:6f1c::1): Ensures the message's destination IP matches either the known IPv4 or IPv6 address for the mosquitto broker.
    - Found by using the filter dns.qry.name contains mosquito



No.	Time	Source	Destination	Protocol	Length	Token	Info
152	2.043380370	10.0.2.3	10.0.2.15	DNS	119		Standard query response 0x4603 AAAA test.mosquitto.org AAAA 2001:41d0:a:6f1c::1 OPT
187	2.088161343	10.0.2.3	10.0.2.15	DNS	107		Standard query response 0x78fa A test.mosquitto.org A 5.196.78.28 OPT
149	2.002037003	10.0.2.15	10.0.2.3	DNS	91		Standard query 0x78fa A test.mosquitto.org OPT
151	2.002503691	10.0.2.15	10.0.2.3	DNS	91		Standard query 0x4603 AAAA test.mosquitto.org OPT
153	2.043580314	127.0.0.53	127.0.0.1	DNS	119		Standard query response 0x0656 AAAA test.mosquitto.org AAAA 2001:41d0:a:6f1c::1 OPT
188	2.088343080	127.0.0.53	127.0.0.1	DNS	107		Standard query response 0xe255 A test.mosquitto.org A 5.196.78.28 OPT
346	5.005122201	127.0.0.53	127.0.0.1	DNS	107		Standard query response 0x732a A test.mosquitto.org A 5.196.78.28 OPT
348	5.005153018	127.0.0.53	127.0.0.1	DNS	119		Standard query response 0x1d27 AAAA test.mosquitto.org AAAA 2001:41d0:a:6f1c::1 OPT
543	7.010505922	127.0.0.53	127.0.0.1	DNS	107		Standard query response 0x9e0b A test.mosquitto.org A 5.196.78.28 OPT
544	7.010585301	127.0.0.53	127.0.0.1	DNS	119		Standard query response 0x7904 AAAA test.mosquitto.org AAAA 2001:41d0:a:6f1c::1 OPT
148	2.001908430	127.0.0.1	127.0.0.53	DNS	91		Standard query 0xe255 A test.mosquitto.org OPT

  - mqtt.retain == 1: Selects messages where the Retain flag is set to true (1).
  - mqtt.qos == 0: Selects messages using Quality of Service level 0 ("At most once").
  - !_ws.malformed: Excludes any malformed packets.

Yields 208 results according to WireShark



Challenge Question 7:

How many MQTT-SN messages on port 1885 are sent by the clients to a broker in the local machine?

Answer: 0

Answer Motivation:

Using filter: mqttsn

- There are no mqttsn messages of any kind in this capture

